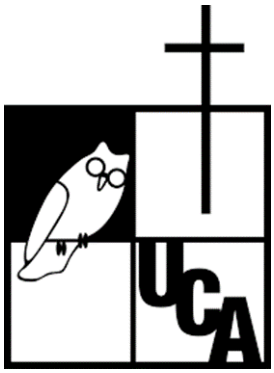
UNIVERSIDAD CENTROAMERICANA JOSÉ SIMEÓN CAÑAS DEPARTAMENTO DE ELECTRÓNICA E INFORMÁTICA



“Informe Tecnico Fase 1”

TEORÍA DE LENGUAJES DE PROGRAMACIÓN

Docente: Jaime Roberto Climaco Navarrete

Integrantes:

Nombre:	**Carnet:**
Axel Jahir Alvarado Alegría	00216022
Oscar Armando Calderón Arguera	00090822
Juan Neftaly Castellanos Hernández	00182222
Adriel Alexander Escobar Gutiérrez	00075522

2 de mayo de 2026

Antiguo Cuscatlán, El Salvador

1. Marco teórico

1.1 La jerarquía de Chomsky

Tipo	Gramática	Reconocedor	Ejemplo relevante
3	Regular	Autómata finito	Identificadores, números, palabras reservadas (el lexer de Mini-C)
2	Libre de contexto (CFG)	Autómata de pila	Paréntesis anidados, expresiones, bloques { } (el parser de Mini-C)
1	Sensible al contexto	Autómata linealmente acotado	{ww}, $\{a^n b^n c^n\}$
0	Sin restricciones	Máquina de Turing	Cualquier lenguaje recursivamente enumerable

Cada clase contiene a la anterior y la inclusión es estricta (Chomsky, 1956, 1959; Hopcroft, Motwani y Ullman, 2006). Un compilador usa Tipo 3 para el análisis léxico y Tipo 2 para el sintáctico; las restricciones de contexto (declarar antes de usar, tipos) se verifican en la fase semántica (Aho et al., 2006).

1.2 Lenguajes formales frente a lenguaje natural

Dimensión	Lenguaje de programación	Español
Estructura	Gramática única y cerrada, definida antes de usarse	Descrita *a posteriori*; léxico abierto y cambiante
Ambigüedad	Se elimina por diseño (precedencia, llaves obligatorias)	Estructural y léxica; es normal, no un defecto
Recursividad	Anidamiento explícito (paréntesis, bloques)	Subordinación y modificación recursivas (*dice que ana piensa que…*)
Dependencias de contexto	Fuera de la gramática (fase semántica)	Concordancia de género y número, concordancia a distancia, correferencia
Gramaticalidad	Binaria: la entrada pertenece o no al lenguaje	Graduada y dependiente del contexto y del hablante

1.3 Qué pueden y qué no pueden hacer las CFG

- **Concordancia con rasgos finitos.** Si género ∈ {m, f} y número ∈ {sg, pl} son rasgos finitos, cada combinación se convierte en un no terminal distinto. La gramática crece, pero sigue siendo Tipo 2.
- **Ambigüedad.** No excede a las CFG, pero una gramática ambigua no puede ser LL(1) (Aho et al., 2006) y obliga a retroceso o a algoritmos tabulares (CYK, Earley, `ChartParser`).
- **Dependencias cruzadas no acotadas.** Sí excederían a las CFG. El lenguaje de copia {ww} no es libre de contexto; su intersección con el regular a*b*a*b* es $\{a^i b^j a^i b^j\}$, que tampoco lo es por el lema de bombeo, y las CFG se cierran bajo intersección con lenguajes regulares (Hopcroft et al., 2006). Shieber (1985) usó un argumento de este tipo con el alemán suizo y Bresnan et al. (1982) discutieron el neerlandés. Joshi (1985) propuso que el lenguaje natural es «suavemente sensible al contexto», entre Tipo 2 y Tipo 1.

1.4 Cómo se lee la evidencia en este informe

Un rechazo del parser admite tres lecturas que no deben confundirse: (1) **lo que demuestra la ejecución**, un comportamiento observado de la gramática implementada; (2) una **limitación del fragmento**, porque falta una regla o una palabra y añadirla mantiene Tipo 2; (3) una **propiedad teórica de las CFG**, que ninguna ejecución sobre oraciones cortas puede probar por sí sola.

Hipótesis de trabajo: un fragmento controlado y finito del español cabe en una CFG con rasgos finitos. Los obstáculos prácticos son la cobertura y la ambigüedad; el de clase es teórico y exige idealizar (k ilimitado).

---

2. Parser formal: Mini-C

Subconjunto. Declaraciones (`int x;`, `int x = e;`), asignación, `if` / `else`, `while`, `print(e);`, bloques, expresiones con `+ - * /`, paréntesis y negación unaria, comparaciones y comentarios `//`. Las llaves son obligatorias en `if` y `while`, lo que elimina el «else colgante». No incluye funciones, otros tipos ni verificación semántica.

Gramática. La BNF, la forma EBNF y la tabla FIRST/FOLLOW están en el Anexo A. Está escrita sin recursión izquierda y factorizada (`Expr → Term ExprR`, `ExprR → + Term ExprR | - Term ExprR | ε`, …). La función `verificar_ll1()` calcula FIRST y FOLLOW sobre el diccionario `GRAMATICA` y reporta **0 conflictos** (Aho et al., 2006). El parser (una función por no terminal, un token de anticipación) se escribió a mano siguiendo esa gramática y no se genera del diccionario; su correspondencia con ella se comprueba mediante las pruebas, no de forma automática.

Implementación (`tlp/formal/`). El lexer usa expresiones regulares (Tipo 3) y trata un carácter ilegal como error. Los bucles `ExprR` y `TermR` se colapsan en nodos binarios asociativos por la izquierda: `8 - 3 - 2` produce `(8 - 3) - 2`. Por eso el árbol de `analizar()` es un árbol de análisis *simplificado*: conserva sentencias, condiciones y operadores, pero no los no terminales auxiliares ni los paréntesis. Se exporta en notación con corchetes, indentado, diccionario y PNG.

Errores. Cada error indica línea y columna. Con `recuperar=True` hay recuperación en modo pánico (sincroniza en `;` o `}`) y se reportan varios errores a la vez.

Entrada	Mensaje del parser
x = 1 +;	Línea 1, col 8: se esperaba número, variable, (o - pero llegó ;
int x = 1 2;	Línea 1, col 11: se esperaba ; pero llegó 2
x = 1 @ 2;	Línea 1, col 7: carácter ilegal @

Pruebas. (i) 12 programas válidos y 13 inválidos en `casos/formal/`; cada inválido declara el fragmento de mensaje esperado y la prueba exige línea y columna válidas. Los inválidos cubren operandos faltantes o seguidos, `;`, paréntesis y llaves faltantes o sobrantes, carácter ilegal, `if` sin paréntesis o sin llaves, asignación a un literal, `int` sin nombre, expresión vacía y `else` huérfano. (ii) Pruebas unitarias de precedencia, asociatividad izquierda, pertenencia del `else` al `if` más cercano, anidamiento de 200 niveles de paréntesis, errores multilínea y recuperación. En total son 28 pruebas; sin spaCy se ejecutan 26 y 2 se omiten (§5).

Relación con Chomsky. Los paréntesis anidados exigen memoria de pila: Mini-C no es regular pero sí libre de contexto (Tipo 2). El parser usa la pila de llamadas como autómata de pila implícito.

---

3. Intento de extensión al español

Se construyó un parser aparte, no una modificación del de Mini-C: aplica la misma idea (CFG y descenso recursivo) con retroceso mediante generadores, para obtener todas las derivaciones (`tlp/espanol/gramatica_es.py`). Cubre un fragmento controlado: léxico de **181 formas**, tercera

persona, presente y pretérito perfecto simple, sujeto explícito. La gramática compacta tiene **10 reglas**, con rasgos de género, número, animacidad y subcategorización verbal.

Fenómeno exigido	Cómo se modela	Resultado
Concordancia de género y número (Det–N–Adj y sujeto–verbo)	Rasgos finitos gen y num comprobados al reducir; cada Adj concuerda con su N	Funciona, incluida la concordancia a distancia y con atractor
Cláusula subordinada	SV → V Que O (completiva: *Pedro dice que Ana piensa que el gato duerme*) y Rel → Que SV (relativa de sujeto)	Funciona para sujeto; no cubre la relativa de objeto
Modificadores	Adj* pospuestos, Adv, SP* (frases preposicionales)	Funciona, pero introduce ambigüedad de adjunción

CFG pura equivalente. Al expandir los rasgos a no terminales se obtiene una CFG sin rasgos de **338 producciones**. El `ChartParser` de NLTK sobre ella coincide con el parser con rasgos en las 13 oraciones probadas (las 10 del catálogo y 3 adicionales; `tests/test_espanol_extension.py`) y en el número de árboles de §4.1. Esto respalda la hipótesis para este fragmento: la concordancia cuesta *tamaño* (10 reglas frente a 338 producciones), no poder expresivo; no prueba la afirmación para todo el español.

---

4. Análisis formal de los puntos de falla

El catálogo ejecutable (`python -m tlp.espanol.catalogo_fallas`) clasifica cada oración en una de cinco clases: **A**, falla del fragmento (falta una regla o una palabra; añadirla mantiene Tipo 2); **B**, límite de la técnica (Tipo 2 pero ambigua, por tanto no LL(1)); **C**, límite de la clase (no libre de contexto, solo como argumento teórico); **D**, fuera de la sintaxis (anomalía semántica); **E**, bien manejada por la gramática.

#	Oración	Fenómeno	Clase (nivel)	Regla o rasgo implicado	Causa del resultado y qué demuestra la ejecución
1	el gato la persigue	pronombre clítico	A (Tipo 2)	SV → V Obj; falta SV → Cl V	Rechazo por regla no escrita
2	come el pan	sujeto tácito	A (Tipo 2)	O → SN SV; bastaría O → SV	Rechazo por regla no escrita
3	Juan y María leen un libro	coordinación nominal	A (Tipo 2)	falta SN → SN y SN	Rechazo por **léxico** (*y* ausente): no prueba nada sobre las CFG
4	la niña que el gato persigue corre	relativa de objeto	A (Tipo 2)	Rel → Que SV (solo hueco en sujeto)	Rechazo por regla no escrita; exigiría categorías con hueco
5	¿dónde está el gato?	interrogativa, copulativo	A (Tipo 2)	sin inversión ni copulativo	Rechazo por **léxico** (*dónde*, *está*): no prueba nada sobre las CFG
6	el hombre ve al gato con el telescopio	adjunción de FP	B (Tipo 2, ambigua)	SN → Det N Adj* SP* Rel? y SV → V Obj SP*	Acepta con **2 árboles**: la ambigüedad se observa
7	el gato que corre persiguen al ratón	concordancia a distancia	E (Tipo 2)	concordancia sujeto–verbo	Rechazo correcto por el rasgo de número
8	los niños de la escuela duerme	concordancia con atractor	E (Tipo 2)	concordancia sujeto–verbo	Rechazo correcto por el rasgo de número
9	Juan y María son rubio y	dependencias	C (argumento	ninguna regla llega a	Rechazo por **léxico** (6

#	Oración	Fenómeno	Clase (nivel)	Regla o rasgo implicado	Causa del resultado y qué demuestra la ejecución
	morena respectivamente	cruzadas	teórico)	aplicarse	formas ausentes): no demuestra ningún límite
10	el libro come la casa	anomalía semántica	D	restricción de selección (semántica)	Acepta con 1 árbol: la anomalía no es sintáctica

Qué demuestra la ejecución y qué no. Un rechazo por vocabulario ausente (filas 3, 5 y 9) o por una regla no escrita (filas 1, 2 y 4) **no** demuestra que una CFG sea insuficiente, porque todos esos casos se pueden ampliar dentro de Tipo 2. Las filas 6, 7, 8 y 10 sí muestran un comportamiento de la gramática ejecutada que sustenta una afirmación.

4.1 Ambigüedad estructural (clase B)

La fila 6 admite dos lecturas: *con el telescopio* modifica a *ve* o a *gato*. Al encadenar frases preposicionales se obtienen 2, 5 y 14 árboles para 8, 11 y 14 palabras (los números de Catalan C_2, C_3 y C_4); el parser propio y NLTK coinciden en los tres conteos (`resultados/fase1_fallas.md`).

Por qué no es LL(1). En `SN → Det N Adj* SP* Rel?` y `SV → V Obj SP*`, un `SP` puede cerrar el `SN` del objeto o continuar en el `SV`; así `FIRST(SP)` pertenece a `FOLLOW(SN)` y la alternativa «otro `SP`» compite con la vacía. Al aplicar `verificar_ll1()` a un fragmento reducido (`SN → Det N SPS`, `SV → V Obj SPS`, `Obj → SN`, `SPS → SP SPS | ε`, `SP → P SN`) se obtiene el conflicto `FIRST(SP SPS) ∩ FOLLOW(SPS) ≠ ∅`. Esa comprobación auxiliar no forma parte del repositorio, pero es reproducible con esas reglas. El parser del español no usa tabla LL(1) sino retroceso. Una CFG entrega *todas* las lecturas pero no elige; elegir exige información semántica o estadística.

4.2 Dependencias cruzadas: «respectivamente» (clase C, argumento teórico)

Qué hizo la ejecución. Rechazó la oración de la fila 9 porque seis formas no están en el léxico. Ese rechazo no tiene valor probatorio: con las palabras y la regla correspondientes, una CFG acepta esa oración concreta.

Qué sostiene el argumento teórico. Con k pares, el patrón $x_1 \ldots x_k\ y_1 \ldots y_k$ con cada x_i concordando con y_i tiene estructura cruzada análoga a la copia {ww}. Como {ww} ∩ a*b*a*b* = $\{a^i b^j a^i b^j\}$ no es libre de contexto (lema de bombeo) y las CFG se cierran bajo intersección con regulares, el fenómeno **para k ilimitado** no sería libre de contexto (Hopcroft et al., 2006; Shieber, 1985). Hay dos salvedades: (1) para k acotado (por ejemplo k ≤ 2) basta una CFG con reglas explícitas, así que ninguna oración corta puede refutar a las CFG; (2) que el español requiera k ilimitado es una idealización, no un hecho empírico establecido. En síntesis: *si* se acepta la idealización, el fenómeno exige más que Tipo 2; con k acotado permanece en Tipo 2. Esta fase no incluye una demostración ejecutable de este límite.

---

5. Contraste con NLTK y spaCy

Las mismas diez oraciones del catálogo se procesaron con tres analizadores (`python -m tlp.apoyo.contraste_nlp`; resultados en `resultados/contraste_nlp.md`).

NLTK se usó como motor CFG independiente (un `ChartParser` sobre la gramática expandida sin rasgos). Coincide con el parser propio en las diez oraciones y en el número de árboles; contrasta dos

formulaciones de la misma clase de gramática. **No es un modelo estadístico** y no analiza nada que la CFG no cubra.

spaCy (Honnibal et al., s. f.) es la comparación con un modelo estadístico. Se ejecutó con spaCy 3.8.18 y `es_core_news_sm` 3.8.0, versiones que el script registra en `resultados/contraste_nlp.md`. Es una dependencia opcional: sin ella sus pruebas se omiten y el reporte queda marcado «No generada».

#	Oración	Parser propio	NLTK	spaCy: raíz / sujeto / objeto
1	el gato la persigue	RECHAZA	RECHAZA	persigue/**NOUN** / gato / —
2	come el pan	RECHAZA	RECHAZA	come/VERB / — / pan
3	Juan y María leen un libro	RECHAZA	RECHAZA	leen/VERB / Juan / libro
4	la niña que el gato persigue corre	RECHAZA	RECHAZA	corre/VERB / niña / que
5	¿dónde está el gato?	RECHAZA	RECHAZA	está/VERB / gato / —
6	el hombre ve al gato con el telescopio	ACEPTA (2 árboles)	ACEPTA (2)	ve/VERB / hombre / gato
7	el gato que corre persiguen al ratón	RECHAZA	RECHAZA	persiguen/VERB / gato / —
8	los niños de la escuela duerme	RECHAZA	RECHAZA	duerme/VERB / niños / —
9	Juan y María son rubio y morena respectivamente	RECHAZA	RECHAZA	rubio/ADJ / Juan / —
10	el libro come la casa	ACEPTA (1 árbol)	ACEPTA (1)	come/VERB / libro / casa

Qué permite concluir, solo sobre estas diez oraciones y este modelo pequeño:

1. spaCy no rechaza ninguna oración, ni siquiera las agramaticales de las filas 7 y 8. Para un modelo estadístico no existe «fuera del lenguaje», solo la mejor estimación. La columna de discordancia del reporte (compara el rasgo `Number` de sujeto y verbo raíz y marca las filas 7 y 8) es una **heurística propia del proyecto**, no una función de spaCy.
2. Donde el parser propio rechaza por alcance (filas 2 a 5), spaCy entrega una estructura; en la fila 4 analiza bien la relativa de objeto (*que* es objeto de *persigue*), que la gramática no cubre.
3. spaCy también puede equivocarse sin avisar: en la fila 1 etiqueta *persigue* como sustantivo. Ante la ambigüedad de la fila 6, la CFG devuelve las dos lecturas y spaCy muestra una sola.

Qué no permite concluir: que un enfoque sea mejor en general ni medir precisión o cobertura; eso exige un conjunto de prueba mayor y métricas (Fase 2).

---

6. Conclusiones y limitaciones

1. **Parser formal.** Una CFG LL(1) basta para un subconjunto de un lenguaje de programación: la gramática no tiene conflictos y las entradas inválidas se rechazan con línea y columna (§2).
2. **Extensión al español.** La concordancia de género y número se resuelve con rasgos finitos dentro de Tipo 2, a costa de tamaño (10 reglas frente a 338 producciones); también se cubren completivas, relativas de sujeto y modificadores (§3).
3. **Puntos de falla y jerarquía.** El lexer es Tipo 3 y el parser de Mini-C, Tipo 2. Los obstáculos observables del español son la cobertura abierta y la ambigüedad; la insuficiencia de clase depende del argumento de dependencias cruzadas con k ilimitado, que es teórico (§1 y §4).
4. **Contraste.** Un reconocedor formal y un modelo estadístico responden preguntas distintas: reconocer pertenencia frente a producir la mejor estructura (§5).

Limitaciones declaradas. (a) El español cubre un fragmento controlado, no es un parser general, y parte de lo que rechaza queda fuera por construcción. (b) El parser de Mini-C está escrito a mano y su correspondencia con la gramática verificada no se comprueba automáticamente; su árbol es

simplificado. (c) El límite de clase (§4.2) no tiene demostración ejecutable. (d) La comparación con spaCy usa un único modelo pequeño y diez oraciones, y no se midió el costo del parser con retroceso. (e) No se evaluó rendimiento cuantitativo (Fase 2).

Pendientes del grupo: nombres de los integrantes, presentación oral con demo en vivo y verificación de la paginación final (§7).

---

7. Reproducibilidad y formato

Instalación y ejecución: ver `README.md`. Las cifras salen de `python run_fase1.py` y de `resultados/`. Con spaCy y el modelo se ejecutan 28 pruebas y todas pasan; sin ellos, 26 y 2 se omiten.

Extensión. El enunciado pide 5–7 páginas y no aclara si portada, referencias o anexos cuentan. Este documento está en Markdown, que no tiene paginación propia. En la revisión del 7-nov-2026 se exportó con `pandoc` a `.docx` (A4, Calibri 11 pt, márgenes de 2,5 cm, tablas a 9 pt) y a PDF con LibreOffice: **7 páginas** con referencias y sin el Anexo A, en el límite superior del rango. Word puede paginar distinto: el grupo debe verificar el PDF final y recortar si supera 7.

Referencias

- Aho, A., Lam, M., Sethi, R. y Ullman, J. (2006). *Compilers: Principles, Techniques, and Tools* (2.ª ed.). Addison-Wesley.
- Bird, S., Klein, E. y Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly (NLTK).
- Bresnan, J., Kaplan, R., Peters, S. y Zaenen, A. (1982). Cross-serial dependencies in Dutch. *Linguistic Inquiry*, 13(4).
- Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3).
- Chomsky, N. (1959). On certain formal properties of grammars. *Information and Control*, 2(2).
- Honnibal, M. et al. (s. f.). *spaCy: industrial-strength NLP in Python* (software). https://spacy.io
- Hopcroft, J., Motwani, R. y Ullman, J. (2006). *Introduction to Automata Theory, Languages, and Computation* (3.ª ed.). Addison-Wesley.
- Joshi, A. (1985). Tree adjoining grammars: how much context-sensitivity is required to provide reasonable structural descriptions? En Dowty, Karttunen y Zwicky (eds.), *Natural Language Parsing*. Cambridge University Press.
- Shieber, S. (1985). Evidence against the context-freeness of natural language. *Linguistics and Philosophy*, 8.